# Project KineticSketch

## What is KineticSketch?

KineticSketch acts as a virtual chemistry lab running in a web browser in which we can type a drug name or write its SMILES notation to instantly gets its 2D drawing (or we can draw manually too), and it will also build a 3D model of that drug, will predict how that molecule wiggles over time and screens it again known compound to find similar drugs and targets. It does it all locally on a normal system without having to install bulky, expensive software and setting up command-line tools.

*This project was developed by Prawin under the guidance of Dr. Rajiv K. Kar, Assistant Professor, Jyoti and Bhupat Mehta School of Health Sciences & Technology (and Associate Faculty, Centre for Nanotechnology), IIT Guwahati.*

## How does it work?

### Step 1: Drawing, Name Search, and Chemical Coding (Input Pipeline)

When we draw a molecule or when we type a drug name, the pipeline translates it into a standard format that computers understand (SMILES). Here is how it is done:

1. Translating a 2D Drawing into Text (SMILES)

   When we draw a molecule on canvas, the computer saves it as simple (x,y) coordinate points for each atom (Carbon, Nitrogen, etc.) and lines (bonds).

   When we click on Render, the browser sends this canvas structure to the server. The server uses RDKit to covert our drawing into a line of text called SMILES (Simplified Molecular Input Line Entry System).

2. Finding Drugs by Name

   If you don't want to draw, you can simply type a drug name like "Aspirin" or "Paracetamol" in the search box and it will automatically gets its SMILES notation by two methods:

   1. Local Offline Search First: It queries a local database on your hard drive (data/drug_database.sqlite) to see if it knows that drug or its synonyms. This database has 2898063 (2.9 mil) drugs. Use the command `sqlite3 data/drug_database.sqlite "SELECT COUNT(*) FROM drugs;"` to verify it.

2. PubChem Online Fallback: If the drug name is rare and not in the local database, it falls back to querying the official National Institutes of Health (NIH) online database (PubChem API) to fetch the SMILES text code. One example - curl -s "https://pubchem.ncbi.nlm.nih.gov/rest/pug/compound/name/aspirin/property/CanonicalSMILES/json"

## Step 2: Building the 3D Shape and Coordinate Optimization

A flat drawing or a SMILES text string doesn't tell us what a molecule actually looks like in 3D space. To figure out the shape, angles, and distances between atoms, the backend processes the structure using RDKit:

1. Adding Hydrogens: In drawings, we usually hide Hydrogen atoms to keep things clean. But for a 3D model, they are essential. The system adds explicit Hydrogens back into the structure.
2. Generating the 3D Coordinates (ETKDGv3): The app uses an algorithm called ETKDGv3 to calculate the starting 3D positions of the atoms. It uses real crystallographic rules (bond lengths and angles mined from actual crystal databases) to shape the molecule. (app/services/cheminformatics.py, lines 105 to 108).
3. Relaxing the Molecule (MMFF94 force field): Just placing atoms in 3D can leave them in unstable or strained positions. The software runs an energy minimization algorithm using the MMFF94 Merck Molecular Force Field. Think of this like letting a metal spring stretch and settle into its most comfortable, relaxed resting shape. (app/services/cheminformatics.py, line 131)
4. Writing Files: Once optimized, the 3D coordinates are saved in three standard formats used by researchers worldwide: SDF, XYZ, and Tripos MOL2 files, making it compatible with other tools.

## Step 3: AI-Powered Molecular Motion Prediction

1. How it reads the molecule: The molecule's 3D coordinate shape is turned into a Position Tensor (an N times 3 matrix, mapping each atom to X, Y, Z). The atom types (Carbon, Nitrogen, etc.) are converted into one-hot numeric features representing the elements (Node Features).
   Predicting the Floppiness (RMSF): The neural network has exactly 1,228,370 parameters with 4 layers and 4 distance scales verify it via .venv/bin/python -c '
   import torch
   from app.services.models import get_predictor
   model = get_predictor(torch.device("cpu"))
   print(f"Total Parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad):,}")

2. It takes the coordinates and predicts the Root-Mean-Square Fluctuation (RMSF) of each atom at two timescales:

   * 10 nanoseconds: How much the atom wiggles in the short term.

* 1 microsecond: How much the atom flexes over a longer, highly dynamic timeframe. This helps researchers instantly see which parts of a drug are rigid (stable binding) and which are floppy.

3. Orientation Independence (Rotation Invariance): No matter how you rotate the molecule in the 3D space, the GNN outputs the exact same wiggling prediction. It accomplishes this by ignoring the absolute coordinates and instead calculating a pairwise distance matrix (measuring how far every atom is from every other atom). It runs this through Gaussian Radial Basis Functions (RBF) to feed the network.

4. The Training Story (Phase A & B): The model was trained in two distinct phases:

    * Phase A (Pre-training on 150,000 synthetic shapes): Taught the GNN basic chemical distances and bond constraints (achieved a Mean Squared Error of 0.062). (scripts/train_mdrepo.py, line 421)

    * Phase B (Fine-tuning on 1,319 real proteins): Fine-tuned on real molecular simulation trajectories from the FastProtFlex dataset (achieved a final validation MSE of 15.039). (The full FastProtFlex dataset contains 3,771 simulations. However, training was performed locally on a consumer AMD Radeon RX 6500M GPU (4 GB VRAM). To prevent Out-Of-Memory (OOM) crashes, we filtered out any complex proteins with more than 1,200 atoms (defined in scripts/train_mdrepo.py at line 429). This left us with exactly 1,319 structures that could fit within our GPU's 4 GB memory limit.)

    * Why is the Phase B error higher? Synthetic structures are relaxed in vacuum (almost no movement, fluctuation value is close to 0). Real proteins in water flex and bend dynamically across a much wider numerical range, so the raw fluctuation scale is naturally much larger.

### Step 4: Drug Repurposing Similarity Search & The RAM Optimization

Once we have our molecule, we want to know: "How chemically similar is it to existing drugs, and what target proteins in the body does it interact with?"

To do this, we compare the input structure against a library of 2,898,063 reference compounds compiled from ChEMBL 37 and DrugCentral:

1. Morgan Fingerprints (ECFP4 equivalent): Both your drawn molecule and the reference drugs are translated into a mathematical Morgan Fingerprint—a list of 2048 binary bits (0s and 1s) representing the local neighborhoods of atoms.
2. The Tanimoto Similarity Score: The app compares the two fingerprints using the Tanimoto formula (calculating the intersection divided by the union of chemical features). A score of 1.0 means they are chemically identical; 0.0 means completely different.

3. The RAM Memory Problem: On disk, the compressed fingerprint file (data/drug_fingerprints.npz) occupies only 219 MB. However, when decompressed into memory as a raw matrix, it expands to 5.5 GB in RAM (shape:2,898,048 rows by

2,048 columns, stored as uint8 integers). Loading this entire matrix into RAM at once causes severe memory lag and page faults on consumer laptops.

How to Verify the File Size on Disk: du -sh data/drug_fingerprints.npz

How to Verify the Matrix Dimensions and Datatype (Shape & Dtype):

.venv/bin/python -c 'import numpy as np; fp = np.load("data/drug_fingerprints.npz"); print(fp["fingerprints"].shape, fp["fingerprints"].dtype)'

4. The Solution (Memory-Mapping & Chunking):

    - Memory-Mapping (mmap_mode='r'): The app loads the file with read-only memory- mapping to let the OS handle paging. (Implemented in app/services/drug_database.py at line 82).
    - Chunked Multiplications: To prevent the computer from freezing, the similarity engine divides the search into blocks of 10,000 compounds at a time. It processes a chunk, discards it from RAM, and moves to the next. This keeps memory usage capped strictly under 80 MB and runs the entire comparison in under a second! (Implemented in app/services/drug_database.py, lines 266 to 282).

5. Target Mapping: Any compound with a Tanimoto similarity score of 0.10 or higher (10% chemical match) (app/services/pdb_repurposing.py at line 102) is cross-referenced with a local dataset of 5,576 drug-target PDB associations (across 5,421 unique PDB targets) to show you which proteins in the body your molecule might bind to.

    How to Verify the Target Counts in SQLite:

    sqlite3 data/drug_database.sqlite "SELECT COUNT(*) FROM drug_targets;"

    sqlite3 data/drug_database.sqlite "SELECT COUNT(DISTINCT pdb_id) FROM drug_targets;"

    Estimating Binding Energy (Delta G):

    The Concept: For each target match, the app estimates Delta G (Gibbs Free Energy of Binding) in kcal/mol. A negative value means the drug binds spontaneously (the more negative the number, the tighter the bond).

    The Scaling Formula: Since running full molecular docking calculations takes too long for a live dashboard, the backend approximates this value using a simple chemical formula based on the similarity score: Delta G = -(Similarity * 9.5 + 2.0) kcal/mol (For example, a 100% match yields a Delta G of -11.50 kcal/mol, whereas a 70% match scales down to -8.65 kcal/mol). This estimation is calculated in app/services/drug_database.py at lines 313-314 inside the target search loop: affinity_kcal = -(similarity * 9.5 + 2.0).

## Step 5: 3D Protein-Ligand Interaction Profiling & PyMOL AI Assistant

Once we look up a drug's target, the app lets us visualize it in 3D and understand the actual physical forces holding the drug in its receptor pocket:

1. Interactive Protein Mapping (PDB Fetching):
    - In the UI: Under the 3D View tab, there is an input field labeled PDB ID and a Fetch button (onclick="fetchPdbTarget()" at app/gui/index.html line 1431).
    - Under the Hood: The app queries the official RCSB Protein Data Bank online API (https://files.rcsb.org/download/{pdb_id}.pdb). To prevent downloading the same files repeatedly, it caches them locally in the scratch/pdb_cache/ directory (Implemented in app/services/pdb_parser.py, lines 9 and 26).
    - In the UI: Once parsed using BioPython (imported in app/services/pdb_parser.py at line 5), a hidden dropdown selector Select Ligand (at app/gui/index.html line1444) appears, listing the bound molecules inside that protein.

2. Interaction Profiling (Calculating Non-Covalent Forces):
    - In the UI: When you select a ligand and click on profile, the app draws dashed lines inside the 3D WebGL viewer connecting the drug to the surrounding amino acids, and renders a detailed table showing the distance of each bond.
    - Under the Hood: The backend measuring script (app/services/interaction_profiler.py at line 69) checks four key forces:
        - Hydrogen Bonds: Checked when atoms are within 3.5 Å of distance and have a binding angle of 130 degree (Implemented in app/services/interaction_profiler.py, lines 161 and 220).
        - Hydrophobic Contacts: Checked when non-polar carbon atoms are within 4.2 Å of distance (Implemented in app/services/interaction_profiler.py, line 466).
        - Pi-Stacking: Parallel rings within 3.3 to 5.8 Å (angle < 35 degree), and perpendicular rings within 4.0 to 7.2 Å (angle 55 degree to 90 degree) (Implemented in app/services/interaction_profiler.py, lines 337 and 359).
        - Salt Bridges: Checked when oppositely charged atoms are within 4.5 Å of distance (Implemented in app/services/interaction_profiler.py, line 263).

3. PyMOL AI Assistant (Natural Language commands):
    - In the UI: The right-hand sidebar has a section titled PyMOL Assistant containing a text box and a Send button (onclick="triggerChatSend()" at app/gui/index.html line 1503).
    - Under the Hood: When you type a query (e.g. "show spheres"), it makes a POST request to /api/chat (handled in app/main.py:910). This calls query_ollama_for_pymol() in app/services/visualizer.py at line 98, which sends the prompt to the local llama-server

(running a local Phi-4-mini GGUF model at my system). The server returns raw PyMOL commands (like show spheres) which are sent back to the browser and executed in the 3Dmol.js WebGL visualizer dynamically.

## Step 6: ADME Drug-Likeness Profiling (Lipinski & Veber Checks)

To be a good medicine, a molecule must be able to survive inside the human body. The app checks your molecule against standard pharmaceutical filters (ADME: Absorption, Distribution, Metabolism, Excretion) to score its drug-likeness.

All these calculations are performed by calling the /api/descriptors endpoint (defined in app/main.py at line 949), which runs the logic in the calculate_adme_descriptors() function inside app/services/descriptors.py (lines 27 to 95):

1. Lipinski's Rule of Five (Oral Absorption): The app checks four physical rules (defined in descriptors.py at lines 49 to 61) to determine if a molecule can be easily absorbed by the gut:
    - Molecular Weight: Must be less than 500 Da (heavier molecules struggle to cross cell walls).
    - LogP (Greasiness): Must be less than 5 (measures if the drug is too soluble in fat vs. water).
    - Hydrogen Bond Donors: Must be 5 or fewer (counts -OH and -NH chemical groups).
    - Hydrogen Bond Acceptors: Must be 10 or fewer (counts Nitrogen and Oxygen atoms).
    - The Pass Limit: A drug molecule is allowed to violate at most one of these four rules to pass the Lipinski filter.

2. Veber's Filter (Oral Bioavailability): This filter checks if the drug can actually enter the bloodstream when taken as a pill (defined in descriptors.py at lines 63 to 71):
    - Rotatable Bonds: Must be 10 or fewer (if a molecule has too many spinning joints, it is too floppy to bind effectively).
    - Polar Surface Area (TPSA): Must be 140 $Å^2$ or smaller (measures the polar surface area of the atoms; if it is too high, it cannot cross cell lipid layers).
    - The Pass Limit: The molecule must pass both of these rules with zero violations to pass the Veber filter.

3. In the UI: The results are rendered dynamically at the bottom of the right-hand panel in the dashboard, showing progress bars and pass/fail indicators next to each metric.

*Note: All verification commands below are for Linux/macOS terminals. On Windows, run these inside WSL2.*